From Saying to Thinking to Doing:

A Practical Guide to LLMs, LRMs, and LAMs

LLMs
Large Language Models: The "Saying" phase of AI.

LRMs
Large Reasoning Models: The "Thinking" phase of AI.

LAMs
Large Action Models: The "Doing" phase of AI.

Shashank Daté
VP of Engineering/Research, Rothbright

Why This Talk, Why Now

The AI Landscape

Three overlapping model categories:

- LLM (2023)
- LRM (2024)
- LAM (2025)

have emerged but are often conflated in media every single week.

The Cost of Confusion

Picking the wrong model type wastes money, increases latency, and erodes trust in AI solutions.

Practical Outcomes

Gain a durable framework, not just a snapshot. No architecture background required; we stay focused on practical application.

LLM Architecture & Generation Loop

Step	What happens
Prompt	Raw user text enters the model
Tokenizer	Splits text into sub-word IDs from a fixed vocabulary
Embeddings	Each token ID becomes a high-dimensional vector
Transformer Stack	N layers refine each token using the surrounding context
Logits	A score for every token in the vocabulary
Sampler	Picks one token — greedy, top-k, or temperature-based
Next Token	Appended to the running sequence; the loop repeats

- One forward pass → one token
- A 500-token answer = 500 passes through the stack
- No hidden scratchpad (→ LRM)
- No actions (→ LAM)

LLMs in Practice

Strengths	Limits
Fast and cheap per call	Hallucinates facts
Broadly capable across domains	Weak at multi-step logic
Massive ecosystem & tooling	No real planning
Cost scales ~linearly with tokens	"Pattern, not proof"

- Definition. A Large Language Model predicts the next token at scale. Trained on text to model the distribution of language. Answers: "What should I say?"
- Where they shine: summarization · translation · classification · rewriting · drafting
- Examples: GPT-4, Claude Sonnet, Llama 3, Gemini Flash, Mistral

LLMs Token Economics

	Input tokens	Output tokens
How they're processed	All at once, in parallel, in one forward pass	One at a time — one pass per token
What you're billed for	Compute to "read" your prompt	Compute to generate every new token
Typical ratio	1X	3–5X structural, not a pricing quirk

Practical implication, in priority order:

1. Trim output length first. Ask for terse formats (JSON, bullets, not prose).
2. Trim input second. RAG smarter, summarize history, use prompt caching.
3. Right-size the model. Don't pay frontier prices for tasks a small model handles.
4. Check out: LLM pricing

LRM Architecture & Reasoning Loop

Step	What happens
Question	The prompt arrives
Reasoning Trace	Model generates hidden "thinking" tokens — long chains of intermediate steps
Self-Check	Model evaluates its own trace — consistent? plausible? complete?
Loop / Branch	If not confident, extend the trace or try a different approach
Final Answer	Visible output — often far shorter than the hidden reasoning

LRM: What is the correct answer, and why?

- Same architecture as an LLM, same transformer stack, same autoregressive loop (the model is talking to itself before talking to you)
- The difference is what gets generated and how much
- Trained via Reinforcement Learning (RL = reward correct reasoning, not just correct answers)
- More thinking tokens → generally better accuracy (to a point & then it overthinks :)

LRMs in Practice

Strengths	Limits
Math, logic, code, multi-step analysis	Slower (seconds to minutes per answer)
Planning steps another system will execute	Pricier — you pay for every thinking token
Justification, not just an answer	Can overthink simple prompts
Higher accuracy on hard problems	Reasoning traces are often hidden or sanitized

LRMs in Practice

- Where they shine: multi-step math · competitive coding · scientific QA · ambiguous problems where justification matters · planning for downstream systems · debugging logic errors
- Examples: OpenAI o1 / o3, DeepSeek-R1, Claude (extended thinking), Gemini Deep Think
- LRMs are the right tool when you need fewer, more correct answers, not more, faster ones.
- "Justification, not just an answer" this is often more valuable than the accuracy bump.
- Some APIs let you cap reasoning budget (thinking.budget_tokens on Claude). Use it. See ***lrm_example.py*** for code.

LRMs in Practice

- A reasoning trace lets humans audit why the model reached a conclusion, which matters for compliance, debugging, and trust.
- Reasoning traces are not a guarantee of correctness the model can produce confident-sounding reasoning that's wrong. Treat the trace as evidence, not proof.
- Hidden vs. visible traces: OpenAI hides most of o1/o3's trace and shows a summary (concerns about leaking training signal). Claude exposes the thinking block. DeepSeek-R1 shows the full chain. Pick a provider based on whether you need to see the reasoning.
- Cost callback: every thinking token is a billable output token - this is why LRMs cost more than LLMs for the same final answer.

LAM Architecture & Action Loop

Step	What happens
Goal	A high-level task arrives — "book me a flight," "fix this bug," "summarize my inbox"
Plan	Model decomposes the goal into steps using an LLM or LRM core
Pick Action / Tool	Model selects a tool to call — API, function, browser click, shell command
Execute	The harness runs the tool — real side effects in the real world
Observe Result	Tool output (success, failure, data, error) is fed back to the model
Loop or Finish	If goal met → finish. Otherwise → re-plan and pick the next action

LAM: What should I do, and how?

- Still the same transformer underneath. The LAM is a loop wrapped around an LLM/LRM - plus tools, memory, and a controller. It is an engineering pattern, not a new model class
- The model never "acts" directly - it emits a tool call (structured output); a **harness** executes it. Hence “harness engineering”.
- Success = a trajectory of correct actions, not a single correct answer.
- Memory lives outside the model - usually a scratchpad (prior tool results), a vector store (long-term recall), or both.
- Error compounds over long horizons. If each step is 95% reliable and the task needs 20 steps, end-to-end success is $0.95^{20} \approx 36\%$. This is why LAMs need approvals, retries, and rollback - they're not magic, they're plumbing.

LAMs in Practice

Strengths	Limits
Automates real workflows end-to-end	Error compounds over long horizons
Operates the systems humans operate	Side effects are real — and sometimes irreversible
Combines reasoning with execution	Needs sandboxing, auth, approvals, rollback
Closes the gap between "answer" and "outcome"	Evaluation is a trajectory, not a score

LAMs in Practice

- Definition. A Large Action Model wraps an LLM or LRM core with tools, memory, and an action loop: calling APIs, clicking buttons, running code, editing files, sending messages.
- Where they shine: software engineering agents · web & desktop automation · RPA over SaaS APIs · data pipeline operations · customer-support agents · robotics & VLAs (vision-language-action)
- Examples: Claude Code, Cursor Agent, Devin · ChatGPT Agent / Operator · browser-use · Rabbit r1 · enterprise RPA agents · physical robots
- The hard part isn't the model, it's everything around it: tool design, auth boundaries, retry logic, observability, rollback, eval harnesses. Building a LAM is 20% prompt engineering, 80% systems engineering.

LLM, LRM, LAM

Dimension	LLM	LRM	LAM
The question	What should I say?	What's true, and why?	What should I do, and how?
Primary output	Text	Text + hidden reasoning trace	Tool calls + actions
Test-time compute	Low — one pass per token	High — many "thinking" tokens before answering	Variable — multiple model calls per task
Side effects	None	None	Yes — real, sometimes irreversible
Typical latency	Milliseconds → seconds	Seconds → minutes	Seconds → hours
Cost driver	Output tokens	Output tokens + thinking tokens	Loops × (input + output) per loop
Failure mode	Hallucinates	Overthinks	Acts wrongly
What it takes to ship	Prompt + cache	Prompt + reasoning budget	Tools + sandbox + auth + audit + rollback

How they are trained:

Stage	LLM	LRM	LAM
1. Pretraining	Next-token prediction on huge text corpora	↑ same	↑ same
2. Supervised fine-tuning (SFT)	Curated input/output pairs teach instruction-following	↑ same	↑ same
3. RLHF (alignment)	RL on human preferences – helpful, harmless, on-tone	↑ same	↑ same
4. Reasoning RL	–	RL rewarding **correct reasoning traces** (verifiable rewards on math, code, logic)	usually included
5. Tool-use / Agentic RL	–	–	RL or imitation on **tool-call trajectories** – reward = task success across many steps

Questions?

Shashank Daté

Email: shashank@rothbright.com

Github: https://github.com/shanko

LinkedIn: https://www.linkedin.com/in/shashankdate/

Demo Time:

Script	Status	Notes
1_llm_example.py	✅ Works	Returned a stakeholder summary. Model added an "honest note" suffix beyond the requested two sentences – that's a Sonnet 4-family stylistic habit, not a bug.
2_lrm_example.py	✅ Works	Both thinking and text blocks rendered correctly. Internal reasoning shows the model deliberating before its No-Go recommendation.
3_lam_example.py	✅ Works	Full agentic loop: 3 tool calls (get_test_results, get_open_bugs, send_email) in correct order, then a final synthesis. Loop terminates correctly when no more tool_use blocks come back.